Proof-of-concept · status update

The problem, in one sentence

We constantly **rewrite** deep-learning code to make it run faster.

But how do we *know* the faster version computes the **same thing** as the original?

- A rewrite that is *almost* identical can silently change the results.
- Silently wrong math $\rightarrow$ wrong predictions, bugs that are very hard to find.

We want a way to be **sure**, not hopeful.

Our example: the "QKV" step inside every Transformer

Attention models — the engine behind modern AI — build three things, **Q**, **K**, **V**, from the input X using three weight matrices.

The clear way (unfused) — three separate matrix multiplications:

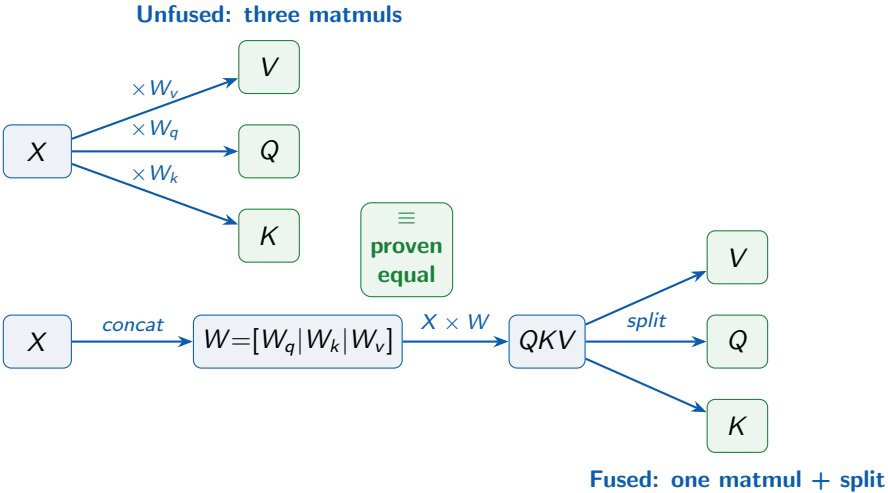
$$Q = X\,W_q \qquad K = X\,W_k \qquad V = X\,W_v$$

The fast way (fused) — glue the weights together, do **one** big multiply, then split:

$$W = [\,W_q \mid W_k \mid W_v\,] \qquad QKV = X\,W \qquad \rightarrow \text{ split into } Q, K, V$$

These **should** give identical results. The question: do they — *always*?

The two programs, side by side



ESBMC proves these two produce **the same result for every input**.

How we check today: we test

We run the program on a **handful of example inputs** and compare.

Like tasting a few spoonfuls of soup and hoping the whole pot is fine.

- Testing only covers the inputs you happened to try.
- A bug hiding in some other input is simply **missed**.
- **Passing tests $\neq$ correct.**

What we actually want: a proof for *all* inputs

Instead of trying a few inputs, check **every possible input at once** — mathematically.

Two possible outcomes:

- ✓ **PROVEN equal** — guaranteed identical for *every* input.
- ✗ **Counterexample** — one specific input where they differ, handed straight to you.

Either way, you learn something **certain**.

ESBMC is an automated *model checker*.

1. It **reads the program** and turns the question *"are these two always equal?"* into one giant logic puzzle.
2. An automated **solver** then either finds an input that breaks it, or proves that **no such input exists**.

You write *no* proofs by hand — it is **push-button**.

The trick that turns a test into a proof

Ordinary test	**Our verification**
one **random** tensor `torch.randn(...)`	a **symbolic** tensor = *any* numbers
checks that one case	checks the **whole range at once**
"looks fine"	a **guarantee**

We assert `unfused == fused`, feed it *any* input, and let ESBMC settle it.

(We keep the numbers in a sane range to rule out oddities like "not-a-number".)

Result: the example is PROVEN correct

- ✓ Proven equal for **all** inputs — and **bit-for-bit exact**, not merely "close".
- ✓ Done **two ways**: a plain-math encoding *and* the real `torch.mm` + `torch.allclose`.
- ✓ When we **deliberately break it** (swap two columns), ESBMC **catches it** and shows the exact failing input.

Whole test suite: **6 / 6** targets behave exactly as expected.

Why formal verification matters here

- **All inputs**, not a few → no hidden corner cases.
- **Exact** → catches tiny numerical drift that testing would wave through.
- **Automatic** → no hand-written proofs, no PhD required to run it.
- **Actionable failures** → it gives you the precise input that breaks things.

For machine learning: you can finally **trust** your fused kernels, rewrites, and compiler optimizations.

A bonus: we made the tool itself stronger

Real PyTorch-style code pushed ESBMC into territory it could not yet handle. So along the way we **found and fixed** the gaps:

- **4 contributions merged** into ESBMC upstream — including a new built-in model for `torch` operations and several bug fixes.

Now *other people* can verify PyTorch-style code too — not just us.

- ✓ Motivating **QKV** example **fully supported** (two encodings, exact proof).
- ✓ Extended to a **second** case: bias-fused linear ($X\,W + b$).
- → **Bigger matrices** and fully native fuse/split — gated on **one** performance improvement upstream.

Takeaway

We can now **prove** — automatically, for **every** input — that a fast PyTorch rewrite computes **exactly** the same result as the original.

Testing can only sample. This is a guarantee.